PROJECT CODE

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <limits>
#include <iomanip>

using namespace std;

// ============================================================
//  STRUCTS
// ============================================================

struct Donor {
   string name;
   string address;
   string contactNumber;

   Donor() = default;
   Donor(const string& n, const string& a, const string& c)
      : name(n), address(a), contactNumber(c) {}
};

struct DonatedItem {
```

```
    string category;
    int    quantity;
    Donor  donor;

    DonatedItem() : quantity(0) {}
    DonatedItem(const string& cat, int qty, const Donor& d)
        : category(cat), quantity(qty), donor(d) {}
};

// ============================================================
//  CLASS
// ============================================================

class DonationPlatform {
private:
    vector<DonatedItem> donatedItems;
    string          fileName;
    string          platformName;

    // — CLEAR all existing data from file ——————————————————
    void clearFile() const {
        ofstream outFile(fileName, ios::trunc);
        if (!outFile) {
            cout << " [Error] Could not clear file.\n";
        }
        outFile.close();
    }
```

```
// — SAVE one item (append) ————————————————————————
void saveToFile(const DonatedItem& item) const {
   ofstream outFile(fileName, ios::app);
   if (!outFile) {
      cout << " [Error] Could not open file for writing.\n";
      return;
   }
   outFile << item.category        << "|"
         << item.quantity        << "|"
         << item.donor.name      << "|"
         << item.donor.address   << "|"
         << item.donor.contactNumber << "\n";
}

// — LOAD all items from file ——————————————————————
void loadFromFile() {
   ifstream inFile(fileName);
   if (!inFile) return;        // first run: no file yet, that's fine

   string line;
   while (getline(inFile, line)) {
      if (line.empty()) continue;

      stringstream ss(line);
      string category, quantityStr, name, address, contact;

      getline(ss, category,   '|');
      getline(ss, quantityStr, '|');
```

```
            getline(ss, name,       '|');
            getline(ss, address,    '|');
            getline(ss, contact);

            int quantity = 0;
            try   { quantity = stoi(quantityStr); }
            catch (...) { quantity = 0; }

            donatedItems.emplace_back(category, quantity,
                                      Donor(name, address, contact));
        }
    }

public:
    // — Constructor ——————————————————————————————
    explicit DonationPlatform(const string& pName  = "Main Platform",
                              const string& fName  = "donations.txt")
        : fileName(fName), platformName(pName)
    {
        // Clear all previous data and start fresh
        clearFile();
        loadFromFile();  // This will load nothing since file is cleared
    }

    // — Safe integer input ———————————————————————————
    int getSafeChoice() const {
        int choice = 0;
        while (true) {
```

```
        if (cin >> choice) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return choice;
        }
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << " [!] Enter a valid number: ";
    }
}

// — Donate ————————————————————————————————————
void donateItem() {
    cout << "\n--- New Donation on [" << platformName << "] ---\n";

    Donor donor;
    cout << "Enter your name         : "; getline(cin, donor.name);
    cout << "Enter your address      : "; getline(cin, donor.address);
    cout << "Enter your contact number : "; getline(cin, donor.contactNumber);

    if (donor.name.empty() || donor.contactNumber.empty()) {
        cout << " [!] Name and contact number cannot be empty. Donation cancelled.\n";
        return;
    }

    string category;
    cout << "Enter item category     : "; getline(cin, category);
    if (category.empty()) {
        cout << " [!] Category cannot be empty. Donation cancelled.\n";
```

```
        return;
    }

    cout << "Enter quantity          : ";
    int quantity = getSafeChoice();
    if (quantity <= 0) {
        cout << " [!] Quantity must be greater than 0. Donation cancelled.\n";
        return;
    }

    DonatedItem item(category, quantity, donor);
    donatedItems.push_back(item);
    saveToFile(item);

    cout << " Donation recorded successfully!\n";
}

// — View all donations ————————————————————————
void viewDonatedItems() const {
    cout << "\n--- Donations on [" << platformName << "] ---\n";

    if (donatedItems.empty()) {
        cout << " No donations recorded yet.\n";
        return;
    }

    cout << left
         << setw(18) << "Category"
```

```
        << setw(10) << "Qty"
        << setw(20) << "Donor Name"
        << setw(25) << "Address"
        << "Contact\n";
    cout << string(90, '-') << "\n";

    for (const auto& item : donatedItems) {
        cout << left
             << setw(18) << item.category
             << setw(10) << item.quantity
             << setw(20) << item.donor.name
             << setw(25) << item.donor.address
             << item.donor.contactNumber << "\n";
    }
}

// — Search donations by category ————————————————
void searchByCategory() const {
    cout << "\nEnter category to search: ";
    string keyword;
    getline(cin, keyword);

    bool found = false;
    cout << "\n--- Search Results ---\n";
    for (const auto& item : donatedItems) {
        if (item.category == keyword) {
            cout << "Qty: "    << item.quantity
                 << " | Donor: " << item.donor.name
```

```
                << " | Contact: " << item.donor.contactNumber << "\n";
            found = true;
        }
    }
    if (!found)
        cout << " No donations found for category: " << keyword << "\n";
}

// ── Display platform name ──────────────────────────
const string& getName() const { return platformName; }
};

// ============================================================
//  MAIN
// ============================================================

int main() {
    DonationPlatform platform("Main Platform", "donations.txt");

    cout << "========================================\n";
    cout << "    DONATION MANAGEMENT SYSTEM\n";
    cout << "========================================\n";
    cout << " NOTE: Previous data has been cleared.\n";
    cout << " Only data from this session will be shown.\n";

    int choice = 0;
    do {
        cout << "\n--- Main Menu [" << platform.getName() << "] ---\n"
```

```
            << "1. Donate an Item\n"
            << "2. View All Donations\n"
            << "3. Search by Category\n"
            << "4. Exit\n"
            << "Enter choice: ";

        choice = platform.getSafeChoice();

        switch (choice) {
            case 1: platform.donateItem();           break;
            case 2: platform.viewDonatedItems();      break;
            case 3: platform.searchByCategory();      break;
            case 4: cout << "\nThank you. for the DONATION !! Goodbye!\n"; break;
            default: cout << " Invalid choice. Please enter 1-4.\n";
        }
    } while (choice != 4);

    return 0;
}
```